

MIVA OPEN UNIVERSITY

Department of Computer Science & Information Technology

University Maintenance Service Request System (MIT 8333)

Comprehensive Project Report & Technical Documentation

E. TESTING AND DEPLOYMENT

E.1 Testing Major Frontend Components

The frontend user interface was systematically tested across three distinct user roles (Student/Staff, Maintenance Officer, and Administrator) to ensure responsive rendering, state persistence, error boundaries, and seamless interactivity:

- **Student Dashboard Portal:** Verified complaint creation form, input validation, priority dropdown selectors, category classification, photo upload previews, request history list, and status badges (PENDING, ASSIGNED, IN_PROGRESS, COMPLETED, CANCELLED).
- **Maintenance Officer Drawer:** Tested work order list filtering, ticket detail drawer expansion, status transition controls (IN_PROGRESS and COMPLETED), and mandatory resolution comment attachments.
- **Administrator Operations Console:** Validated real-time summary cards, category/priority metrics distribution, search bar, ticket routing modals, user creation/editing forms, audit logs, and CSV report export buttons.

E.2 Testing Backend API Endpoints

All backend REST API endpoints were subjected to automated end-to-end integration testing using Node.js execution scripts (tests/system-verification.js). The suite verifies security guards, authentication, role authorization, CRUD operations, transactions, and CSV export functionality:

- **Security & Authentication:** Tested rejection of unauthenticated requests (HTTP 401), registration, password hashing (bcryptSync), JWT cookie issuing, and session validation (/api/auth/me).
- **Complaint CRUD APIs:** Tested POST /api/requests (with photo uploads), GET /api/requests (dynamic search/filter/pagination), and GET /api/requests/[id] (relational details).
- **Routing & State Workflow:** Tested POST /api/assignments (routing requests to officers) and PUT /api/requests/[id]/status (status updates with status log creation).
- **Metrics & Export:** Tested GET /api/reports (summary metrics JSON) and GET /api/reports?export=csv (CSV binary streaming).

E.3 Online Deployment & Database Connectivity

The application was deployed and verified across three production environments: Ubuntu Linux, AWS EC2, and Docker Compose multi-container infrastructure. In all deployments, the application establishes secure connection pools to the containerized MySQL 8.0 database engine, with auto-schema creation and seeding.

F. TECHNICAL DOCUMENTATION

F.1 Introduction and Problem Statement

In higher education institutions, maintaining physical and digital infrastructure—lecture hall HVAC, lab electrical fittings, plumbing, and Wi-Fi access points—is essential. Previously, MIVA Open University relied on informal reporting, causing zero status visibility, manual task routing delays, missing audit trails, and an absence of analytical reporting. The University Maintenance Service Request System digitizes and automates this workflow end-to-end.

F.2 System Objectives

- **Unified Service Portal:** Provide an intuitive web interface for logging complaints with file attachments.
- **Role-Based Access Control:** Enforce strict role separation for Students/Staff, Officers, and Admins.
- **Relational Database Engine:** Utilize MySQL 8.0 with connection pooling and lazy SQLite fallback.
- **Cryptographic Security:** Use scryptSync password hashing and HMAC-SHA256 signed JWT cookies.
- **Reporting & Audit:** Generate real-time analytics and downloadable CSV reports.

F.3 Requirement Analysis

- **Functional Requirements:** User auth, request creation, status tracking, task routing, officer resolution, user management, audit logging, and CSV export.
- **Non-Functional Requirements:** Sub-200ms latency, responsive glassmorphism UI, containerized portability, and zero-downtime database auto-seeding.

F.4 Frontend Technologies Used

- **Framework & UI:** Next.js 16 (App Router paradigm) with React 19 Server and Client Components.
- **Styling Architecture:** Vanilla CSS Modules featuring custom CSS tokens (variables.css), glassmorphism dark-mode styling (main.css), priority badges, and responsive CSS grids.

F.5 Backend Technologies Used

- **Runtime & API:** Node.js v22 execution engine with Next.js App Router API Route Handlers (/api/*).
- **Security & Middleware:** Node native crypto module for scryptSync password hashing (16-byte random salt, 64-byte key length), HMAC-SHA256 JWT signatures, and Next.js Edge Middleware (src/middleware.ts).

F.6 Database Architecture & Relationships

The system utilizes a relational MySQL 8.0 database engine driven by the mysql2/promise library with connection pooling. The schema supports strict relational integrity and cascading operations:

- **User & Role (1-to-Many):** Role table defines permissions; referenced by User.roleId.
- **Request & Category (Many-to-1):** RequestCategory categorizes service complaints via Request.categoryId.
- **Request & Creator (Many-to-1):** User creates service complaints via Request.creatorId.
- **Assignment (1-to-1 / CASCADE):** Assignment links Request.id to User.officerId and User.assignedById with ON DELETE CASCADE.
- **StatusLog (1-to-Many / CASCADE):** StatusLog records timestamped state changes (previousStatus, newStatus, comment) with ON DELETE CASCADE.

F.7 API Documentation & Endpoint Reference

The backend provides a complete RESTful API suite accepting JSON and Multipart Form payloads:

Method	Endpoint	Role Required	Description
POST	/api/auth/register	Public	Registers new Student/Staff account
POST	/api/auth/login	Public	Authenticates user and issues HTTP-only JWT cookie
GET	/api/auth/me	Authenticated	Returns session profile of authenticated user
GET	/api/requests	Authenticated	Lists complaints (filtered by role, category, status)
POST	/api/requests	Student/Staff	Submits new complaint with optional photo upload
GET	/api/requests/[id]	Authenticated	Fetches ticket details, assignments, and audit logs
PUT	/api/requests/[id]/status	Officer / Admin	Updates ticket status (IN_PROGRESS, COMPLETED)
POST	/api/assignments	Admin	Routes unassigned ticket to Maintenance Officer
GET	/api/users	Admin	Lists system users or filters Maintenance Officers
GET	/api/reports	Admin	Returns metrics JSON summary or downloads CSV file

F.8 Testing Evidence (Automated Integration Log)

Executing `node tests/system-verification.js` produces 100% passing results:

```
=====
0>ÿ STARTING SYSTEM VERIFICATION & API TEST SUITE
=====
' [PASS] Security: Unauthenticated API access rejected (401)
' [PASS] Auth: Student / Admin / Officer logins valid (200)
' [PASS] Requests: Student submits complaint & lists tickets (201/200)
' [PASS] Assignments: Admin routes request to Maintenance Officer (200)
' [PASS] Workflow: Officer updates ticket to IN_PROGRESS & COMPLETED (200)
' [PASS] Reports: Admin fetches metrics & exports CSV file (200)
=====
0<B TEST SUITE COMPLETED: 13 PASSED, 0 FAILED (100% Success Rate)
```

F.9 Deployment Information

The application supports multiple automated and containerized deployment pipelines:

- **Automated Linux Script (deploy.sh):** 1-click deployment script with Docker Engine auto-installer, HOST_IP detection, and health checks.
- **Docker Compose Multi-Container:** Containerized web app (miva_web_app) and MySQL 8.0 (miva_mysql_db) with named volumes (mysql_data, uploads_data).
- **AWS EC2 & Amazon ECR:** AWS cloud hosting with ECR image repository, Nginx reverse proxying, and PM2 process management.

F.10 Challenges Encountered & Solutions

- **SQLite Concurrency Locks:** Resolved by introducing mysql2 connection pooling and lazy SQLite fallback instantiation.
- **Turbopack Config Deprecation:** Removed deprecated eslint block from next.config.ts for Next.js 16 compatibility.
- **Docker Container Ownership:** Configured Dockerfile to grant explicit chown -R nextjs:nodejs /app ownership for non-root execution.

F.11 Conclusion

The University Maintenance Service Request System (MIT 8333) satisfies all academic, technical, security, testing, and deployment requirements. By integrating Next.js 16, MySQL 8.0, glassmorphism UI design, automated testing, and Docker containerization, the platform delivers an enterprise-ready solution for MIVA Open University.

